

# LaTeX Report for Python GUI Application

Shubham Sahu

April 5, 2025

## 1 Overview

This report provides a comprehensive analysis of a Python GUI application implemented in `pyGUI.py`. It includes:

- Detailed explanations for each function in the source code.
- A flowchart representing the application workflow.

## 2 Detailed Explanation of Functions

### 2.1 FileTab Class

**`__init__(self, file_path='')`**

Initializes a new file tab by setting up instance variables for the file path and LaTeX components. It creates UI elements (such as the list of components, preview area, and a statistics label) and, if a file path is provided, loads the LaTeX file immediately.

**`initUI(self)`**

Constructs the user interface for a file tab by establishing a two-panel layout:

- **Left Panel:** Contains a search box and a list widget for displaying document components.
- **Right Panel:** Displays a live preview (rendered in HTML) and a statistics label indicating the number of components selected.

**`load_tex_file(self, file_path)`**

Loads the LaTeX file from the provided path. It saves the file path, parses the file to extract LaTeX components, and populates the component list in the UI.

**`parse_tex_file(self, filepath)`**

Reads the LaTeX file and splits it into the preamble and the document body. It uses regular expressions to identify elements (such as title and author) and creates a “Title Page” component if these are found. It then calls helper functions to extract sections and environments.

**`extract_sections(self, body)`**

Utilizes regular expressions to extract sections and subsections within the document body. Hierarchical titles are created (with subsections prefixed by their parent section), and the corresponding content is stored.

**`extract_environments(self, body)`**

Searches for common LaTeX environments (such as `abstract`, `figure`, `table`, and `equation`) in the document body. Each found environment is saved as a component with its complete LaTeX content.

### **populate\_component\_list(self)**

Populates the list widget with the names of the extracted components and applies appropriate styling (e.g., color coding for the preamble, bold font for sections, and indentation for subsections).

### **filter\_components(self)**

Filters the component list based on user input in the search box, showing only those components whose names match the search text (case insensitive).

### **update\_report(self)**

Triggered when the user selects components from the list. This function retrieves the selected components, updates the statistics (number of components and estimated pages), and generates a markdown preview of the report (with each component's title and LaTeX code) that is converted to HTML for display.

### **get\_selected\_components(self)**

Returns the list of currently selected component names. This list is later used during report generation to determine which components to include in the final PDF.

## **2.2 PreviewWindow Class**

### **\_\_init\_\_(self, parent=None)**

Sets up a simple, read-only window for displaying the live HTML preview of the report. This preview window can be toggled on and off based on user interaction.

## **2.3 LatexReportGenerator Class**

### **\_\_init\_\_(self)**

Initializes the main application window, configures the overall layout, sets up a tab widget for managing multiple LaTeX files, and connects signals (such as those updating the preview) to their corresponding functions.

### **initUI(self)**

Creates the user interface for the main window, including:

- A control panel with buttons for adding a new LaTeX file and exporting the report.
- A tab widget to manage multiple file tabs.
- A status bar for displaying messages (e.g., confirmations or error messages).

### **button\_style(self)**

Returns a CSS-like style string for the buttons in the application to ensure a consistent and modern appearance.

### **add\_new\_file(self)**

Opens a file dialog to allow the user to select a LaTeX file. When a file is chosen, it creates a new FileTab, adds it as a new tab in the tab widget, and updates the status bar with a confirmation message.

### **close\_tab(self, index)**

Closes the currently open tab. If it is the last tab, a “Welcome” tab with an introductory message is loaded instead of leaving the interface empty.

**show\_tab\_context\_menu(self, position)**

Displays a context menu when the user right-clicks on a tab, offering options such as closing or renaming the tab.

**rename\_current\_tab(self)**

Renames the current tab by appending “(Modified)” to its name. This could be extended to allow custom names via a dialog box.

**toggle\_preview(self)**

Toggles the visibility of the preview window. If not already open, the preview window is created and displayed; if it is open, it is hidden.

**update\_preview\_content(self, content)**

Updates the HTML content in the preview window. This function is connected to a signal so that changes in the report (e.g., component selection) are immediately reflected.

**generate\_report(self)**

Generates the final PDF report by:

- Checking that a LaTeX file is loaded and at least one component is selected.
- Creating a new LaTeX document using the `pylatex` package.
- Iterating over the selected components and appending their LaTeX code to the document (with special handling for the title page).
- Prompting the user for a save location and generating the Tex.
- Updating the status bar with a success message and optionally providing a button to open the generated Tex.